Highlighted Source

The blocks below exercise the token kinds the two highlighters divide differently.

Ruby

```ruby
# frozen_string_literal: true
require 'json'

module Billing
  DEFAULT_RATE = 0.075
  $audit = []

  class Invoice < Struct.new(:number)
    include Comparable
    attr_reader :lines

    def initialize(number, currency: :eur)
      @number = number
      @currency = currency
      @lines = []
    end

    def total(rate = DEFAULT_RATE)
      amount = @lines.sum { |line| line[:amount] }
      raise ArgumentError, "empty\n" if amount.zero?
      amount * (1 + rate)
    end

    def to_s
      "Invoice #{@number}: #{total} #{@currency}"
    end
  end
end

invoice = Billing::Invoice.new(7)
puts invoice.total
```

Python

```python
"""Module docstring."""
from dataclasses import dataclass


```

```python
@dataclass
class Ledger:
    name: str
    entries: list[int]

    def balance(self, opening: float = 0.0) -> float:
        # running total
        total = opening
        for value in self.entries:
            total += value * 1.5
        return None if total == 0 else total
```

JavaScript

```javascript
const RATE = 0.075;
const PATTERN = /^[a-z]+\d{2,}$/gi;

export class Ledger extends Base {
  constructor(name, entries = []) {
    super();
    this.name = name;
  }

  get total() {
    return this.entries.reduce((a, b) => a + b, 0) * (1 + RATE);
  }

  toString() {
    return `Ledger ${this.name}: ${this.total}`;
  }
}
```

C

```c
#include <stdio.h>
#define MAX_LINES 64

struct ledger {
  int total;
};

static int sum_lines(const int *values, size_t count) {
  int total = 0;
  for (size_t i = 0; i < count; i += 1) total += values[i];
  return total;
```

```
}
```

TypeScript

```
export function convert(doc: string): string {
  const total: number = doc.length;
  return `${doc}:${total}`;
}
```